

2進数・浮動小数点

2進数とは

C系列の言語仕様の授業の際にメモリーは0と1で出来ていると話をしたと思います。これを2進数といいます。

私達が普段使っている数字は、

0.1.2.3.4.5.6.7.8.9の10進数を使っています。

1桁目で9まで数えられる、2桁目で10の2乗の100-1まで数えられるものです。

2進数とは

それに対して2進数は、0・1の2つのみで、
1桁目で1、2桁目で3、3桁目で7
n桁目で2のn乗-1まで数えられる数値です。

読んでもわからないと思うので実際に比較してみましょう。

2進数と10進数の比較

2進数	10進数
-----	------

0 0 0 0	0
---------	---

0 0 0 1	1
---------	---

0 0 1 0	2
---------	---

0 0 1 1	3
---------	---

0 1 0 0	4
---------	---

0 1 0 1	5
---------	---

2進数	10進数
-----	------

1 0 1 0	10
---------	----

1 0 1 1	11
---------	----

1 1 0 0	12
---------	----

1 1 0 1	13
---------	----

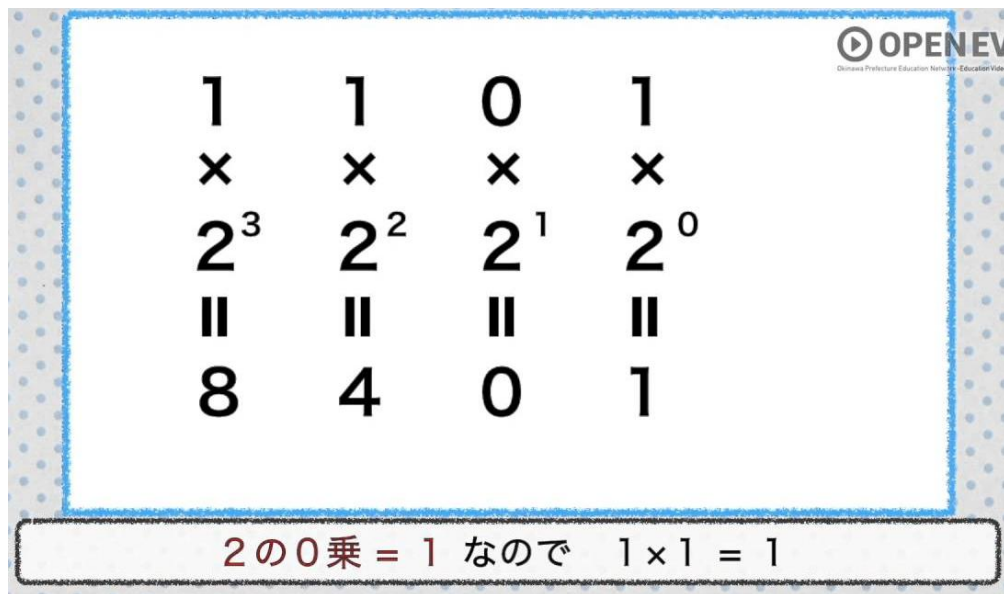
1 1 1 0	14
---------	----

1 1 1 1	15
---------	----

2進数→10進数への変換

たまに就職試験のSPIや会社のテストなどで出てくるので出来るようにしましょう。

2進数を10進数に治す方法は



1	1	0	1
×	×	×	×
2^3	2^2	2^1	2^0
8	4	0	1

2の0乗 = 1 なので $1 \times 1 = 1$

数値の下に*2を書きます。
その2の指数を右から順番に0、1、2とします。
それらの計算を行います。
出た数字を足し算します。

$$8 + 4 + 0 + 1 = 13$$

2進数 1101 = 10進数 13
になります。

2進数・浮動小数点

10進数 → 2進数の変換

逆に10進数を2進数に治す方法はひたすら2で割ります。

2	)	13	
2	)	6	... 1
2	)	3	... 0
2	)	1	... 1
2	)	0	... 1

今度はあまりを下から並べると
1 1 0 1 になります。

これが、10進数の13を
2進数に直したときの値になります。

問題

10進数 15732を2進数に直してみよう。

2進数 1010011を10進数になおして見よう。

2進数の小数

10進数 1 0 0 1 0 1 0.1 0.0 1

10倍 10倍 10分の1 10分の1

2進数 1 0 0 1 0 1 0.1 0.0 1

2倍 2倍 2分の1 2分の1

1の2分の1という事は、
2進数の0.1は
10進数の0.5になる

2進数・浮動小数点

2進数 → 10進数 (小数ver)

$$\begin{array}{ccccccccc} 0 & . & 1 & 1 & 0 & 1 & & & \\ * & & * & & * & & * & & * \\ \hline 1 & & 1 & & 1 & & 1 & & 1 \\ 2^0 & & 2^1 & & 2^2 & & 2^3 & & 2^4 \end{array}$$

2進数から10進数にする場合はほとんど同じ方法で出来ます。

←の式を解いて

$$0.5 + 0.25 + 0 + 0.0625 = 0.8125$$

2進数の0.1101 は
10進数の0.8125 だと分かります。

2進数・浮動小数点

10進数 → 2進数 (小数ver)

10進数の小数部分を2進数に治す際は逆に2掛けます。
その後1以上になったら1を取り出します。

$2^* \quad 0.8125$

$2^* \quad 0.625 \dots 1$

$2^* \quad 0.25 \dots 1$

$2^* \quad 0.5 \dots 0$

$2^* \quad 0.0 \dots 1$



今度は取り出した1を上から
並べると
0.1101なので、

10進数の0.8125は
2進数で0.1101

2進数の小数の欠点

小数の2進数は先程のように割り算で求めていくため、循環小数になってしまうものがあります。

例えば10進数の0.1を2進数にしようとする、
0.00011001100110011...となり途中から1001をループしてしまいます。

そのため、C#では丸め処理と言われるものが行われ、微小な端数がズレてしまいます。

2進数の足し算・引き算

2進数の足し算引き算は普通にそのままでできます。

$$\begin{array}{r} 10101 \\ + 11101 \\ \hline 110010 \end{array}$$

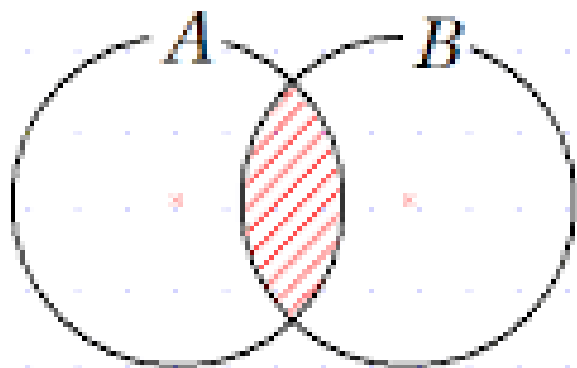
$$\begin{array}{r} 10101 \\ - 111 \\ \hline 1110 \end{array}$$

2進数の積集合・和集合

そもそも積集合・和集合とは対象Aと対象Bの集まりのことです。

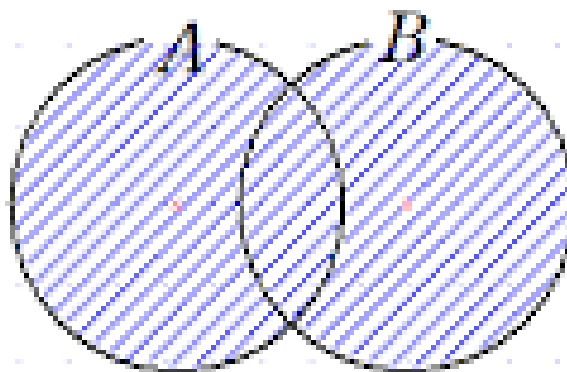
$A \cap B$ (AかつB)

両方みtas



$A \cup B$ (AまたはB)

少なくとも一方をみtas
(両方も可)



2進数の積集合・和集合

2進数では集合の計算を行えます。

積集合（かつ、&）は
どちらも1の場合のみ
1になります。

$$\begin{array}{r} 0 1 0 1 \\ \& 1 1 0 0 \\ \hline 1 0 1 0 0 \end{array}$$

和集合（または、|）は
どちらかが1の場合に
1になります。

$$\begin{array}{r} 0 1 0 1 \\ | 1 1 \\ \hline 1 0 1 1 1 \end{array}$$

2進数と集合の活用

この2進数の考え方と集合の考えが結構な頻度でプログラムに出現しますので、一緒に見てみましょう。

4 個の参照

```
enum Effect : int
```

```
{
```

```
    None = 0,
```

```
    // 状態異常
```

```
    Mahi = 0x_0000_0001, // 麻痺
```

```
    Doku = 0x_0000_0010, // 毒
```

```
    Nemuri = 0x_0000_0100, // 眠り
```

```
    Yakedo = 0x_0000_1000, // やけど
```

```
    Hirumi = 0x_0001_0000, // 怯み
```

```
    Noroi = 0x_0010_0000, // 呪
```

```
    // 状態検知
```

```
    /* 動けないグループ */
```

```
    DontMove = Mahi | Nemuri | Hirumi | Noroi, // 0x_0011_0101
```

```
    /* ダメージを受けるグループ */
```

```
    Damage = Doku | Yakedo | Noroi, // 0x_0010_1010
```

```
}
```

初めて見るかもしれませんが、
プログラム上に2進数を入力することが出来ます。
2進数の入力には0xとつけると2進数扱いになります。

Enumの値を階段状に1が被らないように決めます。
その後グループ用のEnumも宣言します。

2進数と集合の活用

この2進数の考え方と集合の考えが結構な頻度でプログラムに出現しますので、一緒に見てみましょう。

```
static void Main(string[] args)
{
    Effect effect = Effect.None;

    if((effect & Effect.DontMove) == 0)
    {
        // 動ける
    }

    if ((effect & Effect.Damage) >= 0)
    {
        // ダメージを受ける
    }
}
```

今のeffectの値とDontMoveの積集合に一つも1がなければifの中の処理を行う。
麻痺、睡眠、怯み、呪のいずれかだと、
積集合に1がつく

今のeffectの値とDamageの積集合に一つでも1があればif文の処理を行う。
毒、やけど、呪のいずれかだと、
積集合に1がつく

2進数と集合の活用

```
0 個の参照
class Program
{
    0 個の参照
    static void Main(string[] args)
    {
        /*bool ですべての状態上を管理する場合*/
        bool isMahi = true;
        bool isDoku = true;
        bool isNemuri = true;
        bool isYakedo = true;
        bool isHirumi = true;
        bool isNoroi = true;

        /*Enumで管理する場合*/
        Effect effect = Effect.None;
        effect |= Effect.Mahi;
        effect |= Effect.Doku;
        effect |= Effect.Nemuri;
        effect |= Effect.Yakedo;
        effect |= Effect.Hirumi;
        effect |= Effect.Noroi;

        /* enumを戻す場合 */
        effect &= ~Effect.Mahi;
    }
}
```

このEnumでの管理を使えると
変数も少なくすることが出来ます。

すべての状態異常で一つ一つ
boolを作る必要がなくなり、
一つで管理することが出来ます。

浮動小数点

プログラムで小数を扱う際に内部でどのような処理をしているかの解説になります。

大前提ですが、float型やdouble型も0.1のみで出来ており、小数なんてものは1 bitには入りません。

浮動小数点

一旦わかりやすくするために10進数で考えます。

先ほどもありましたが、メモリには小数点がありません。
なので、123.456みたいな小数を表す際にまず、
この数字を下記のような式に変換します。

$$123456 * 10^{-3}$$

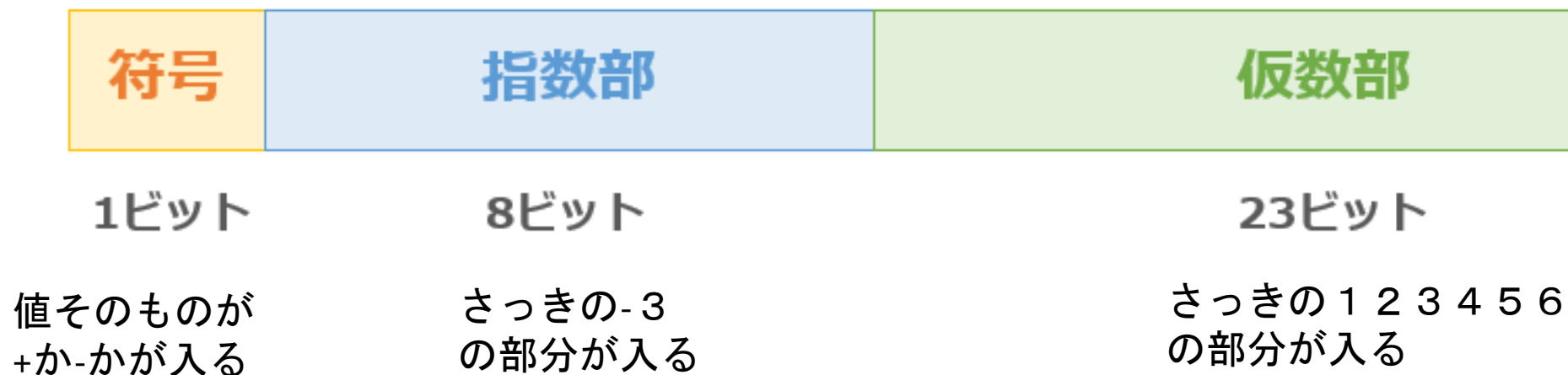
そしてメモリとしては123456と-3だけを記録します。
その値を使う際は取り出す直前に上の計算をすることで
小数を使えるようにしています。

2進数・浮動小数点

浮動小数点

float型は32bitを下記のように分割して使用します。

32ビット形式の浮動小数点数（IEEE754形式）



2進数・浮動小数点

浮動小数点

では2進数で考えていきましょう。

10進数の123.625を2進数に直しましょう。

整数部分 1 2 3 → 1 1 1 1 0 1 1

小数部分 6 2 5 → 1 0 1

1 2 3 . 6 2 5 → 1 1 1 1 0 1 1 . 1 0 1

これを1. ○○○ * 2^n の形にします。

1. 1 1 1 0 1 1 1 0 1 * 2^6

2進数・浮動小数点

浮動小数点

ここにさっき出した1.の後ろから全て入ります。

1 1 1 0 1 1 1 0 1

余った桁数は後ろに全部0が入ります。



ここにさっき出した

2^6 の6が入ります。がこの8ビットで-127～127までの数値を入れるため、0が127の値になるようになっています。そのためここには6に127を足した、133が入ります。

float型の限界

int型が約2億程度まで入ることは皆さん知っていると思います。
int型は32ビットから符号を除いた31ビットをフルに使用しているため、10桁ものデータを入れる事ができますが、float側には指数部分がありデータを入れる幅が小さくなってしまっています。そのため、float型の有効桁数は7桁となっています。

※2進数の小数の絡みもあります。



float型の限界

唯、桁数を指定出来るので実はint型より大きい数字が入ります。

実に、約 $2 * 2^{127}$ 3 4 0 0 澗（かん）まで入ります。

唯、有効数字の問題でMax使うと4 0 0 穰くらい丸められます

日本の国家予算が1 2 2 兆円くらいなので、
3 千京年分の国家予算くらいが適当な数字になります。

※ 万・億・兆・京・垓・秭・穰・溝・澗・正・載・極
恒河沙・阿僧祇・那由他・不可思議・無量大数